

PROJECT PORTFOLIO

건파이어 리본

DirectX 11 개인 FPS에서 맵틀·내비메시 바이너리 저장, 트레일·파티클·후처리, 애니메이션과 전투 시스템을 구현했다.



FPS 전투 장면 · 렌더링 효과와 UI

프로젝트	개인 프로젝트
담당 역할	렌더링 효과, 내비게이션 메시, 애니메이션, UI, 게임플레이 로직
기술	C++, DirectX 11, HLSL, ImGui, JSON, FMOD

<https://yollo4179.github.io/projects/gunfire-reborn/>

내가 구현한 영역

렌더링 효과, 내비게이션 메시, 애니메이션, UI, 게임플레이 로직

DirectX 11로 만든 개인 FPS 프로젝트다. 플레이어가 이동하며 원거리·근거리 몬스터와 보스를 상대하는 전투를 구성했다. 전투 화면을 만드는 렌더링 효과뿐 아니라 맵툴, 내비게이션 메시, 애니메이션 전환, UI, 리소스 로딩까지 한 흐름으로 구현했다.

맵툴은 셀과 배치 데이터를 바이너리로 저장한다. 런타임은 저장한 셀로 이동을 판정하고, 객체별 동적 버퍼로 트레일과 파티클을 갱신한다. 애니메이션·UI·로딩은 각각의 매니저와 데이터 구조로 연결했다.

담당 구현

- **맵툴과 이동:** 지형 광선 선택, 삼각형 셀·몬스터·충돌체 배치, 클라이언트용·맵툴용 바이너리 저장, 셀 이웃 연결, 이동 판정과 경계 슬라이딩.
- **렌더링:** 객체별 트레일 버퍼와 Catmull-Rom 보간, 파티클 인스턴싱, 카툰 외곽선, 글로우, 디졸브, 물 디스토션.
- **게임 런타임:** 애니메이션 전환과 보간, UI 관리, 몬스터·보스 전투, 오브젝트 풀링, 로딩 스레드, 바이너리 모델 데이터 읽기.

기술 구성

C++, DirectX 11, HLSL, ImGui, JSON, FMOD

실행 화면과 구현 코드에 연결된 블로그·영상 링크는 PDF에서 클릭해 열 수 있습니다.

02 / 구현 목차

기능별 구현

01	맵틀의 삼각형 편집과 내비메시 바이너리 저장	04
02	맵틀의 몬스터·충돌체 배치와 실행 데이터 저장	08
03	이웃 셀 이동과 경계 슬라이딩	10
04	객체별 동적 버퍼와 Catmull-Rom 트레일	14
05	파티클을 인스턴스 버퍼로 묶기	18
06	노멀·깊이 렌더 타겟으로 카툰 외곽선 그리기	21
07	다운샘플링과 분리 블러로 글로우 합성하기	25
08	노이즈 임계값과 경계 색으로 디졸브 구현하기	28
09	화면 UV와 노이즈로 물 디스토션 구현하기	30
10	현재 자세에서 다음 애니메이션으로 전환하기	32
11	UI 태그 등록과 팝업 Z 순서·이벤트 관리	36
12	FBX 모델과 애니메이션을 실행용 바이너리로 변환하기	39
13	작업 함수와 스레드로 리소스 로딩 나누기	43

01 / 맵 제작과 이동

맵툴의 삼각형 편집과 내비메시 바이너리 저장

지형을 찍어 만든 삼각형 셀을 두 종류의 .dat 파일로 저장하고, 클라이언트가 셀과 스테이지 번호를 읽도록 구성했다.

[블로그 원문 ↗](#)

구현 구조

TEXT

```
MapTool/Private/Level_GamePlay.cpp
Engine/Private/Navigation.cpp
Engine/Private/Cell.cpp
```

지형 선택 → 정점 스냅 → 삼각형 방향 정리 → 셀·스테이지 저장 → 런타임 셀 생성과 이웃 연결.

정점을 스냅하고 삼각형 방향 맞추기

맵툴은 선택한 점이 기존 점의 범위 안에 들어오면 같은 좌표를 사용한다. 삼각형의 AB와 BC를 외적하고 Y 성분의 부호로 B-C 순서를 정리한다. 셀의 변 방향이 정해져야 런타임에서도 같은 기준으로 안쪽과 바깥쪽을 판정할 수 있다.

MapTool/Private/Level_GamePlay.cpp 발췌

CPP

```
void CLevel_GamePlay::Adjusting_Triangle(TRIANGLE_VERTICES& Triangle)
{
    _vector vectorAB = XMLoadFloat3(&(Triangle.vPointB)) - XMLoadFloat3(&(Triangle.vPointA));
    _vector vectorBC = XMLoadFloat3(&(Triangle.vPointC)) - XMLoadFloat3(&(Triangle.vPointB));
    _vector vecCross = XMVector3Cross(vectorAB, vectorBC);
    if (XMVectorGetY(vecCross) < 0)
    {
        _float3 vTempPoint;
        vTempPoint = Triangle.vPointB;
        Triangle.vPointB = Triangle.vPointC;
        Triangle.vPointC = vTempPoint;
    }
}
```

편집 데이터와 실행 데이터 저장

클라이언트용 내비메시 파일에는 셀 수, 세 정점, 스테이지 번호를 기록한다. 맵툴용 파일은 편집할 때 다시 사용할 점 목록도 함께 기록한다. 런타임은 삼각형을 `cCell`로 만들고 공통 변을 찾아 이웃 인덱스를 연결한다.

구분	저장 정보	사용처
클라이언트	셀 수·삼각형 좌표·스테이지	셀 생성과 이동 판정
맵툴	셀 데이터·편집 점 목록	삼각형 편집 재개

맵틀의 삼각형 편집과 내비메시 바이너리 저장 · 계속

Engine/Private/Navigation.cpp 발췌

파일 열기 실패 처리의 한계: 아래 원본의 `0 == hFile`은 `CreateFile`의 실패 반환값을 검사하지 못한다. 실패 분기는 `hFile == INVALID_HANDLE_VALUE`로 수정해야 한다. 발췌에는 원본 구현을 그대로 표시했다.

C++

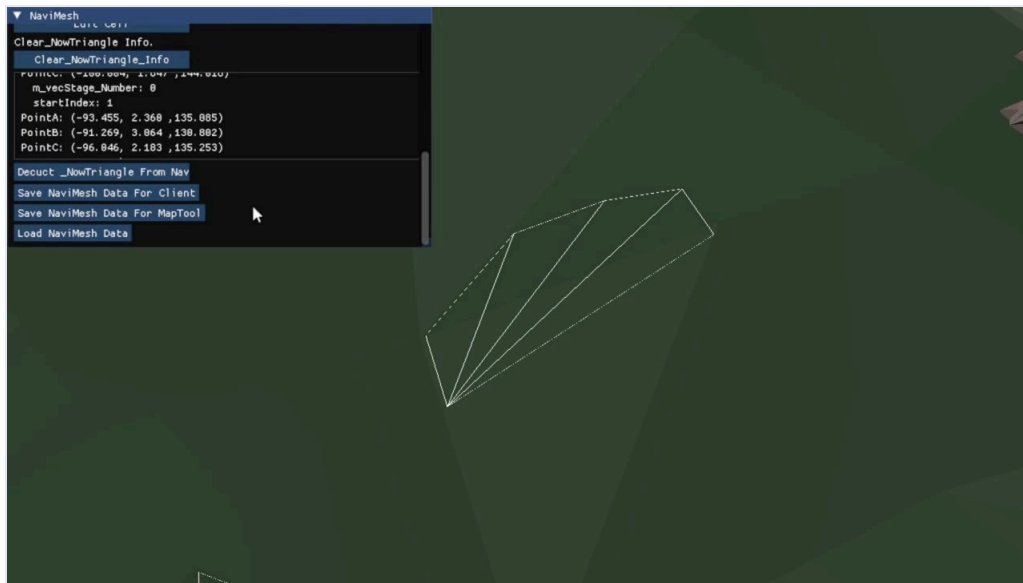
```
HRESULT CNavigation::Initialize_Prototype(const _tchar* pNavigationDataFile)
{
    _ulong          dwByte = {};
    HANDLE          hFile = CreateFile(pNavigationDataFile, GENERIC_READ, 0, nullptr, OPEN_EXISTING, FILE_ATTRIBUTE
_NORMAL, 0);
    if (0 == hFile)
        return E_FAIL;
    int iNum = { 0 };
    ReadFile(hFile, &iNum, sizeof(_int), &dwByte, nullptr);
    while (true)
    {
        _float3      vPoints[3] = {};
        int iStage = { 0 };
        ReadFile(hFile, vPoints, sizeof(_float3) * 3, &dwByte, nullptr);
        ReadFile(hFile, &iStage, sizeof(_int), &dwByte, nullptr);
        if (0 == dwByte)
            break;
        CCell* pCell = CCell::Create(m_pDevice, m_pContext, vPoints, m_Cells.size(), iStage);
        if (nullptr == pCell)
            return E_FAIL;
        m_Cells.push_back(pCell);
    }
    CloseHandle(hFile);
    if (FAILED(Setup_Neighbors()))
        return E_FAIL;
}
```

C++ · 계속

```
#ifdef _DEBUG
    m_pShader = CShader::Create(m_pDevice, m_pContext, TEXT("../EngineSDK/hlsl/Shader_Cell.hlsl"), VTXP::Elemen
ts, VTXP::iNumElements);
    if (nullptr == m_pShader)
        return E_FAIL;
#endif
    return S_OK;
}
```

맵틀의 삼각형 편집과 내비메시 바이너리 저장 · 계속

편집 장면과 저장 자료



맵틀에서 내비메시 삼각형을 만들고 클라이언트용·맵틀용 저장 메뉴를 사용하는 장면 · MP4 직접 열기

```
typedef struct ENGINE_DLL
{
    XMFLOAT3 vPointA;
    XMFLOAT3 vPointB;
    XMFLOAT3 vPointC;
} TRIANGLE_VERTICES;
```

삼각형 셀 정점과 방향 정의

```
void (Level_GamePlay::Creating_Vertex())
{
    pair<_vector, _vector> Ray =Get_PickingPosInWorld();
    _vector vRayPos = Ray.first;
    _vector vRayDir = Ray.second;
    _float fDist = 0;
    for (auto& Triangle : m_Triangles)
    {
        _vector vPointA = DirectX::XMLoadFloat3(&Triangle.vPointA);
        _vector vPointB = DirectX::XMLoadFloat3(&Triangle.vPointB);
        _vector vPointC = DirectX::XMLoadFloat3(&Triangle.vPointC);

        if (TriangleTests::Intersects(
            vRayPos, vRayDir,
            XMVectorSetW(vPointA, 1.f),
            XMVectorSetW(vPointB, 1.f),
            XMVectorSetW(vPointC, 1.f),
            fDist))
        {
            _vector PickingPos = fDist * vRayDir + vRayPos;

            _float4 PickingPosStore;
            XMStoreFloat4(&PickingPosStore, PickingPos);

            if (0 == m_iNowTriangle_Point)
            {
                XMStoreFloat3(&m_NowTriangle.vPointA, PickingPos);
                Mapping_Points(m_NowTriangle.vPointA);
                m_iNowTriangle_Point += 1;
                return;
            }
            else if (1 == m_iNowTriangle_Point)
            {
                XMStoreFloat3(&m_NowTriangle.vPointB, PickingPos);
                Mapping_Points(m_NowTriangle.vPointB);
                m_iNowTriangle_Point += 1;
                return;
            }
            else if (2 == m_iNowTriangle_Point)
            {
                XMStoreFloat3(&m_NowTriangle.vPointC, PickingPos);
                Mapping_Points(m_NowTriangle.vPointC);
                m_iNowTriangle_Point += 1;
                return;
            }
        }
    }
}
```

마우스 광선과 지형의 교점 계산

맵틀의 삼각형 편집과 내비메시 바이너리 저장 · 계속

```

ImGui::SliderInt("NowPoint:", &m_iNowTriangle_Point, 0, 2);
if (3 <= m_iNowTriangle_Point)
{
    if (ImGui::Button("Add_Vertices To Navigation as a Cell", ImVec2(200, 20)))
    {
        Adjusting_Triangle(m_NowTriangle);
        m_pBaseTerrain->Add_Triangle_To_Navigation(m_NowTriangle);
        m_NaviMeshTriangles.push_back(m_NowTriangle); //저장중
        m_vecStage_Number.push_back(m_iNowStageNumber);

        m_vecPoints_Navimesh.push_back(m_NowTriangle.vPointA);
        m_vecPoints_Navimesh.push_back(m_NowTriangle.vPointB);
        m_vecPoints_Navimesh.push_back(m_NowTriangle.vPointC);
        m_NowTriangle = {};
        m_iNowTriangle_Point = 0;
        m_bCreateMode = false;
    }
}

```

정점 순서 조정

```

void CLevel_GamePlay::Adjusting_Triangle(TRIANGLE_VERTICES& Triangle)
{
    _vector vectorAB = XMLoadFloat3(&(Triangle.vPointB)) - XMLoadFloat3(&(Triangle.vPointA));
    _vector vectorBC = XMLoadFloat3(&(Triangle.vPointC)) - XMLoadFloat3(&(Triangle.vPointB));
    _vector vecCross = XMVector3Cross(vectorAB, vectorBC);
    /* 반시계방향이었다면 시계방향으로 바꿔준다. */
    if (XMVectorGetY(vecCross) < 0)
    {
        float3 vTempPoint;
        vTempPoint = Triangle.vPointB;
        Triangle.vPointB = Triangle.vPointC;
        Triangle.vPointC = vTempPoint;
    }
}
/* 찍은점이 일정거리 가깝다면 근사시킨다. */
void CLevel_GamePlay::Mapping_Points(_float3& Point)
{
    for (auto Point_Stored : m_vecPoints_Navimesh)
    {
        _float distSquare =
            (Point_Stored.x - Point.x) * (Point_Stored.x - Point.x) +
            (Point_Stored.y - Point.y) * (Point_Stored.y - Point.y) +
            (Point_Stored.z - Point.z) * (Point_Stored.z - Point.z);

        if (distSquare < 1.f)
        {
            Point = Point_Stored;
            return;
        }
    }

    return;
}

```

인접 정점 위치 보정

```

if (ImGui::Button("Save NaviMesh Data For Client"))
{
    HANDLE hFile = CreateFile(L"..\\Bin\\Resources\\LoadData\\Client_NaviMesh_Stage01.dat", GENERIC_WRITE, 0, 0, CREATE_ALWAYS, FILE_ATTRIBUTE_NORMAL, 0);

    if (INVALID_HANDLE_VALUE == hFile) {
        MSG_BOX("File Open :Failed");
        return;
    }

    _ulong dwByte(0);
    _ulong dwSize(0);

    int iSize = m_NaviMeshTriangles.size();

    WriteFile(hFile, &iSize, sizeof(_int), &dwByte, 0);
    for (_int i = 0; i < iSize; ++i)
    {
        WriteFile(hFile, &m_NaviMeshTriangles[i], sizeof(TRIANGLE_VERTICES), &dwByte, 0);
        WriteFile(hFile, &m_vecStage_Number[i], sizeof(_int), &dwByte, 0);
    }

    MSG_BOX("Completed Save!");
    CloseHandle(hFile);
}

```

클라이언트용 내비메시 바이너리 저장

02 / 맵 제작과 이동

맵툴의 몬스터·충돌체 배치와 실행 데이터 저장

몬스터와 충돌체를 편집하고, 편집용 데이터와 클라이언트용 데이터를 서로 다른 바이너리 형식으로 저장한다.

[블로그 원문 ↗](#)

구현 구조

TEXT

MapTool/Private/Level_GamePlay.cpp

오브젝트 종류 선택 → 위치·크기·회전 편집 → 내비메시 인덱스 지정 → 편집용·클라이언트용 저장.

종류별 몬스터 배치 데이터

맵툴은 몬스터 종류별로 배치 목록을 관리한다. 클라이언트용 저장은 종류 수와 종류별 개수를 먼저 기록하고, 각 객체의 크기·회전각·위치·내비메시 인덱스를 순서대로 쓴다. 실행 시 배치에 필요한 값이 하나의 레코드가 된다.

MapTool/Private/Level_GamePlay.cpp 발췌

CPP

```
if (ImGui::Button("Save Monsters For Client Data"))
{
    HANDLE hFile = CreateFile(m_iNowMapPaths_Client[(int)m_eCurStage][(int)CREATE_OBJECT_MODE::OBJECT], GENERIC_WRITE, 0, 0, CREATE_ALWAYS, FILE_ATTRIBUTE_NORMAL, 0);
    if (INVALID_HANDLE_VALUE == hFile) {
        MSG_BOX("File Open :Failed");
        return;
    }
    _ulong dwByte(0);
    _ulong dwSize(0);
    WriteFile(hFile, &m_iMonsterNum, sizeof(_int), &dwByte, 0);
    for (_int i = 1; i <= m_iMonsterNum; ++i)
    {
        _int iNowMonsterNum = m_Monsters[i].size();
        WriteFile(hFile, &iNowMonsterNum, sizeof(_int), &dwByte, 0);
    }
    for (int i = 1; i <= m_iMonsterNum; ++i)
        for (int j = 0; j < m_Monsters[i].size(); ++j)
        {
            WriteFile(hFile, &(m_Monsters[i][j]->m_Desc.vScale), sizeof(_float3), &dwByte, 0);
            WriteFile(hFile, &(m_Monsters[i][j]->m_Desc.fAngle), sizeof(_float), &dwByte, 0);
            WriteFile(hFile, &(m_Monsters[i][j]->m_Desc.vPosition), sizeof(_float4), &dwByte, 0);
            WriteFile(hFile, &(m_Monsters[i][j]->m_Desc.iNaviIndex), sizeof(_int), &dwByte, 0);
        }
    MSG_BOX("Completed Save!");
}
```

CPP · 계속

```
    CloseHandle(hFile);
}
```

편집 화면과 실행 파일의 역할

ImGui에서 위치와 크기, 회전을 바꾸면 해당 오브젝트의 트랜스폼에 반영한다. 몬스터는 배치 위치에 해당하는 내비메시 셀 인덱스도 보관한다. 충돌체는 `CUBE_DESC`로 관리하며, 편집용 몬스터 파일은 `MONSTER_DESC`를 기록한다.

데이터	역할
<code>vScale</code>	배치 크기
<code>fAngle</code>	회전각
<code>vPosition</code>	배치 위치
<code>iNaviIndex</code>	이동 판정을 시작할 셀

삼각형의 저장과 이웃 연결은 [내비메시 바이너리](#)에서 이어진다.

03 / 맵 제작과 이동

이웃 셀 이동과 경계 슬라이딩

현재 셀과 이웃 셀을 따라 최종 위치를 검사하고, 연결이 없는 경계에서는 이동 벡터를 투영한다.

[블로그 원문 ↗](#)

구현 구조

TEXT

Engine/Private/Navigation.cpp
Engine/Private/Cell.cpp
Engine/Private/Transform.cpp

이동 후보 위치 → 현재 셀 검사 → 이웃 셀 탐색 → 경계 방향으로 이동 벡터 투영 → 이동 가능 여부 재검사.

현재 셀에서 이웃 셀로 이동하기

`isMove()`는 월드 위치를 내비메시 로컬 공간으로 변환한다. 현재 셀 밖이면 변에 연결된 이웃을 따라가고, 위치를 포함한 셀을 찾았을 때 현재 인덱스를 갱신한다. 이웃 인덱스가 -1이면 이동을 허용하지 않는다.

Engine/Private/Navigation.cpp 발췌

CPP

```
_bool CNavigation::isMove(_fvector vWorldPos)
{
    _vector          vLocalPos = XMVector3TransformCoord(vWorldPos, XMMatrixInverse(nullptr, XMLoadFloat4x4(&m_World
Matrix)));
    _int              iNeighborIndex = { -1 };
    if (true == m_Cells[m_iCurrentIndex]->isIn(vLocalPos,iNeighborIndex))
    {
        return true;
    }
    else
    {
        if(-1 ==iNeighborIndex)
            return false;
        while (true)
        {
            if (-1 == iNeighborIndex)
                return false;
            if (true == m_Cells[iNeighborIndex]->isIn(vLocalPos, iNeighborIndex))
            {
                m_iCurrentIndex = iNeighborIndex;
                return true;
            }
        }
    }
    return true;
}
```

이웃 셀 이동과 경계 슬라이딩 · 계속

CPP · 계속

}

경계 방향으로 슬라이딩하기

`Get_Elements_For_Sliding()`은 현재 위치를 기준으로 변의 시작점과 방향을 반환한다. 트랜스폼은 변에 대한 투영점과 셀 중심 방향을 구한 뒤 이동 벡터의 경계 성분을 계산한다. 마지막에는 새 위치에 `isMove()`를 다시 적용한다.

Engine/Private/Transform.cpp 발췌

CPP

```
pair<_vector , _vector> SlidingPairs = pNavigation->Get_Elements_For_Sliding_FromCurCell(vPosition);
_vector vLineAB = SlidingPairs.second;
_vector vPointA= SlidingPairs.first;
_vector vCenter = (XMLoadFloat3(&(pNavigation->Get_A_Point_From_CurCell(CCell::POINT_A))) +
    XMLoadFloat3(&(pNavigation->Get_A_Point_From_CurCell(CCell::POINT_B))) +
    XMLoadFloat3(&(pNavigation->Get_A_Point_From_CurCell(CCell::POINT_C)))
    ) / 3.f;
vCenter = XMVectorSetW(vCenter, 1.f);
_vector vLineAC = vCenter -vPointA;
_vector vLineAD = XMVector3Normalize(vLineAB) * XMVector3Dot(XMVector3Normalize(vLineAB),vLineAC);
_vector vPointD = vPointA + vLineAD;
_vector vNormalForSliding = XMVector3Normalize(vCenter - vPointD);
_vector vSlidingVector = {};
_vector vRes = XMVector3Dot(vNormalForSliding, vLineAB);
if (!XMVectorEqual(vLineAB, _vector()).m128_f32[0]) {
    vSlidingVector = vLook + vNormalForSliding * (XMVector3Dot(vNormalForSliding, -vLook));
    vSlidingVector = XMVector3Normalize(vLineAB) * XMVector3Dot(XMVector3Normalize(vLineAB), vSlidingVector);
}
vPositionBySliding =
    vPosition + XMVector3Normalize(vSlidingVector) * fTimeDelta * 10;
```

이동 장면과 벡터 계산

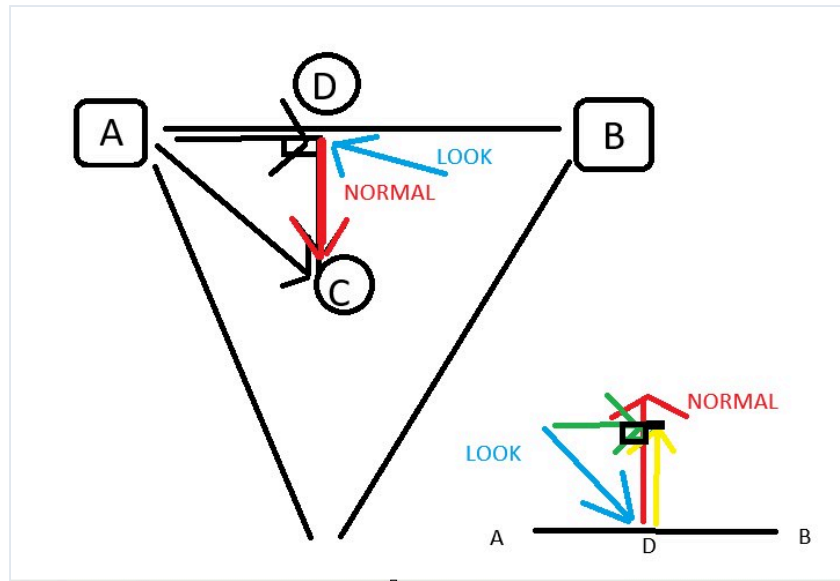
```
int iNum = { 0 };
ReadFile(hFile, &iNum, sizeof( int ), &dwByte, nullptr);
while (true)
{
    _float3 vPoints[3] = {};
    int iStage = { 0 };

    ReadFile(hFile, vPoints, sizeof( float3 ) * 3, &dwByte, nullptr);
    ReadFile(hFile, &iStage, sizeof( int ), &dwByte, nullptr);
    if (0 == dwByte)
        break;
    //세 정점과 인덱스
    CCell* pCell = CCell::Create(m_pDevice, m_pContext, vPoints, m_Cells.size(), iStage);
    if (nullptr == pCell)
        return E_FAIL;

    m_Cells.push_back(pCell);
}
CloseHandle(hFile);
if (FAILED(Setup_Neighbors()))
    return E_FAIL;
```

공유 변을 기준으로 이웃 셀 등록

이웃 셀 이동과 경계 슬라이딩 · 계속



셀 경계와 캐릭터 이동 방향

```

if (pNavigation) {
    _vector vPosition = pNavigation->SetUp_HeightOnNavigation(Get_State(CTransform::STATE_POSITION));
    /*슬라이딩 벡터 구하기*/
    /*C= Position Of Objects*/
    pair<_vector, _vector> SlidingPairs = pNavigation->Get_Elements_For_Sliding_FromCurCell(vPosition);
    _vector vLineAB = SlidingPairs.second;
    _vector vPointA = SlidingPairs.first;
    _vector vCenter = (XMLoadFloat3(&(pNavigation->Get_A_Point_From_CurCell(CCell::POINT_A))) +
        XMLoadFloat3(&(pNavigation->Get_A_Point_From_CurCell(CCell::POINT_B))) +
        XMLoadFloat3(&(pNavigation->Get_A_Point_From_CurCell(CCell::POINT_C)))
        ) / 3.f;
    vCenter = XMVectorSetW(vCenter, 1.f);

    _vector vLineAC = vCenter - vPointA;

    _vector vLineAD = XMVector3Normalize(vLineAB) * XMVector3Dot(XMVector3Normalize(vLineAB), vLineAC);
    _vector vPointD = vPointA + vLineAD;

    _vector vNormalForSliding = XMVector3Normalize(vCenter - vPointD);
    _vector vSlidingVector = {};

    _vector vRes = XMVector3Dot(vNormalForSliding, vLineAB);

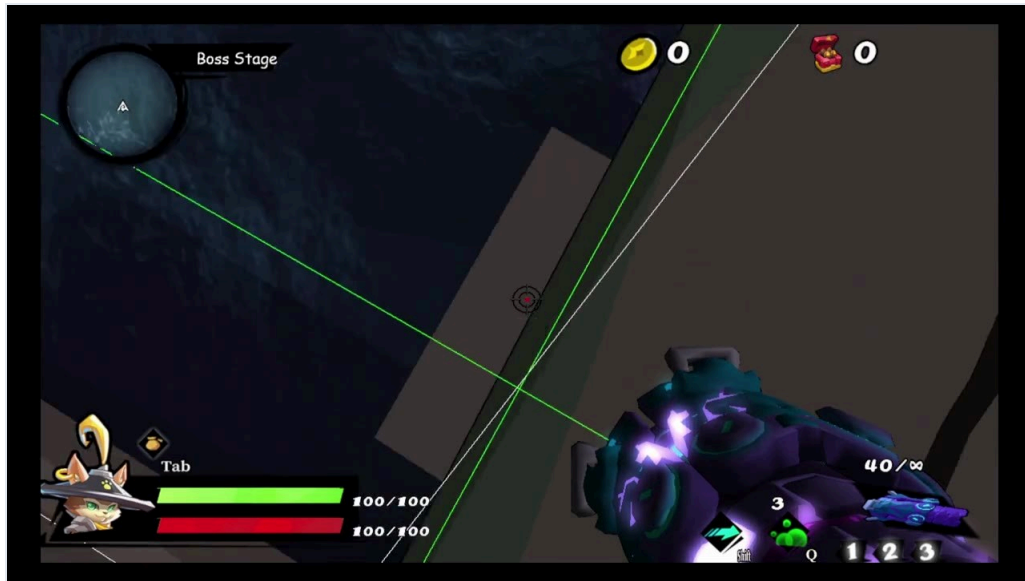
    if (!XMVectorEqual(vLineAB, _vector()).m128_f32[0]) {
        vSlidingVector = vLook + vNormalForSliding * (XMVector3Dot(vNormalForSliding, -vLook));
        vSlidingVector = XMVector3Normalize(vLineAB) * XMVector3Dot(XMVector3Normalize(vLineAB), vSlidingVector);
    }

    vPositionBySliding =
        vPosition + XMVector3Normalize(vSlidingVector) * fTimeDelta * 10;
    /*슬라이딩 벡터 유효한지 확인*/
    _bool isSlidingAvailable = pNavigation->isMove(vPositionBySliding);
}

```

경계 방향에 이동 벡터를 투영하는 계산

이웃 셀 이동과 경계 슬라이딩 · 계속



내비메시 경계와 이동 방향을 표시한 슬라이딩 시연 · MP4 직접 열기

04 / 트레일과 화면 효과

객체별 동적 버퍼와 Catmull-Rom 트레일

복제 인스턴스마다 동적 정점 버퍼를 만들고, 끝점 쌍을 Catmull-Rom으로 보간해 무기의 궤적을 그린다.

[블로그 원문 ↗](#)

구현 구조

TEXT

Engine/Private/VIBuffer_Trail.cpp
Client/Private/Trail_Monster02.cpp

복제 인스턴스의 버퍼 생성 → 끝점 두 개 변환 → 제어점 보관 → Catmull-Rom 보간 → UV·GPU 버퍼 갱신.

트레일마다 동적 정점 버퍼 생성

Clone()은 복제본의 Initialize()를 호출한다. 초기화에서는 TRAIL_DESC의 최대 정점 수로 버퍼 크기를 정하고 CPU 쓰기가 가능한 동적 정점 버퍼를 만든다. 각 트레일의 m_TrailVertices가 해당 객체의 궤적을 보관한다.

Engine/Private/VIBuffer_Trail.cpp 발췌

CPP

```
ZeroMemory(&m_BufferDesc, sizeof m_BufferDesc);
m_BufferDesc.ByteWidth = m_iVertexStride * m_iNumVertices;
m_BufferDesc.BindFlags = D3D11_BIND_VERTEX_BUFFER;
m_BufferDesc.Usage = D3D11_USAGE_DYNAMIC;
m_BufferDesc.StructureByteStride = m_iVertexStride;
m_BufferDesc.CPUAccessFlags = D3D11_CPU_ACCESS_WRITE;
m_BufferDesc.MiscFlags = 0;
VTXPOSTEX* pVertices = new VTXPOSTEX[m_iNumVertices];
ZeroMemory(pVertices, sizeof(VTXPOSTEX) * m_iNumVertices);
ZeroMemory(&m_SubResourceDesc, sizeof m_SubResourceDesc);
m_SubResourceDesc.pSysMem = pVertices;
if (FAILED(__super::Create_Buffer(&(this->m_pVB))))
    return E_FAIL;
```

두 줄의 정점을 곡선으로 연결

Add_Points_CatMullRom()은 위·아래 끝점을 각각 보관한다. 네 제어점 사이에 중간 정점 쌍을 넣고, 쌓인 정점 수에 맞춰 UV와 삼각형 인덱스 수를 갱신한다. 정점 한도를 넘으면 앞부분을 정리하고 새 궤적을 이어 쓴다.

객체별 동적 버퍼와 Catmull-Rom 트레일 · 계속

Engine/Private/VIBuffer_Trail.cpp 발췌

CPP

```

for (_uint i = 0; i < m_iCatmullRomCnt; ++i)
{
    _uint index = i * 2 + m_iVerticesCnt - 2;
    _float fWeight = (_float)(i + 1) / (m_iCatmullRomCnt + 1);
    _vector vPosHigh = XMVectorCatmullRom
    (
        XMVectorSetW(XMLoadFloat3(&m_TrailVertices[m_iCatmullRomIndex[0]].vPosition), 1.f),
        XMVectorSetW(XMLoadFloat3(&m_TrailVertices[m_iCatmullRomIndex[1]].vPosition), 1.f),
        XMVectorSetW(XMLoadFloat3(&m_TrailVertices[m_iCatmullRomIndex[2]].vPosition), 1.f),
        XMVectorSetW(XMLoadFloat3(&m_TrailVertices[m_iCatmullRomIndex[3]].vPosition), 1.f),
        fWeight
    );
    XMStoreFloat3(&m_TrailVertices[index].vPosition, vPosHigh);
    _vector vPosLow = XMVectorCatmullRom
    (
        XMVectorSetW(XMLoadFloat3(&m_TrailVertices[m_iCatmullRomIndex[0] + 1].vPosition), 1.f),
        XMVectorSetW(XMLoadFloat3(&m_TrailVertices[m_iCatmullRomIndex[1] + 1].vPosition), 1.f),
        XMVectorSetW(XMLoadFloat3(&m_TrailVertices[m_iCatmullRomIndex[2] + 1].vPosition), 1.f),
        XMVectorSetW(XMLoadFloat3(&m_TrailVertices[m_iCatmullRomIndex[3] + 1].vPosition), 1.f),
        fWeight
    );
    XMStoreFloat3(&m_TrailVertices[index + 1].vPosition, vPosLow);
}

```

정점의 좌표와 렌더 행렬 연결

입력 행렬로 변환한 끝점을 정점 버퍼에 기록한다. `CTrail_Monster02::Bind_ShaderResources()`의 기본 렌더 경로는 단위 월드 행렬을 바인딩해 정점에 기록된 위치를 사용한다. 위치 갱신은 `Map()`과 `Unmap()`으로 GPU 버퍼에 전달한다.

트레일 실행과 보관 자료



무기 소켓의 이동을 따라 트레일 정점이 이어지는 장면

객체별 동적 버퍼와 Catmull-Rom 트레일 · 계속

```
void CVIBuffer_Trail::Add_Points_CatMullRom(_fmatrix vtxTrail_High_Matrix, _fmatrix vtxTrail_Low_Matrix)
{
    m_iEndIndex = m_iCatmullRomCnt * 2 + m_iVerticesCnt;
    m_iCatmullRomIndex[2] = m_iEndIndex - 2;
    m_iCatmullRomIndex[3] = m_iEndIndex;
}
```

트레일 제어점과 보간 대상 설정

```
if (m_iVerticesCnt <= 6) //CatmullRom보간이 가능할 때까지 정점을 삽입합니다.
{
    _vector vLocalTrailLow = XMLoadFloat3(&m_vLocalTrailLow);
    _vector vLocalTrailHigh = XMLoadFloat3(&m_vLocalTrailHigh);
    XMStoreFloat3(&m_TrailVertices[m_iVerticesCnt].vPosition, XMVector3TransformCoord(vLocalTrailLow, vtxTrail_Low_Matrix));
    XMStoreFloat3(&m_TrailVertices[m_iVerticesCnt + 1].vPosition, XMVector3TransformCoord(vLocalTrailHigh, vtxTrail_High_Matrix));
    m_iVerticesCnt += 2;
}
```

보간에 필요한 정점 수 확인

```
else {
    if (true == m_bStartSpline) //보간을 시작해야하는 지점(인덱스를 재설정)- 1.정점 수 재조정 / 2.처음 보간을 시작할 때
    {
        m_iCatmullRomIndex[0] = m_iVerticesCnt - 6;
        m_iCatmullRomIndex[1] = m_iVerticesCnt - 4;
        m_bStartSpline = false;
    }
    if (m_iEndIndex + 2 >= m_iNumMaxVertices)// 정점 수가 임계치 이상 도달 시 정점 수를 1/4 만큼 줄인다.
    {
        _uint iRemoveCnt = m_iVerticesCnt / 4;
        if (0 != iRemoveCnt % 2)iRemoveCnt -= 1;
        m_iVerticesCnt -= iRemoveCnt;
        for (_uint i = 0; i < m_iVerticesCnt; i += 2)
        {
            m_TrailVertices[i].vPosition = m_TrailVertices[iRemoveCnt + i].vPosition;
            m_TrailVertices[i + 1].vPosition = m_TrailVertices[iRemoveCnt + i + 1].vPosition;
        }
        m_bStartSpline = true;
        return;
    }
}
```

최대 정점 수와 오래된 정점 처리

```
m_TrailVertices[m_iEndIndex - 2].vPosition = m_TrailVertices[m_iVerticesCnt - 2].vPosition;
m_TrailVertices[m_iEndIndex - 1].vPosition = m_TrailVertices[m_iVerticesCnt - 1].vPosition;
_vector vLocalTrailLow = XMLoadFloat3(&m_vLocalTrailLow);
_vector vLocalTrailHigh = XMLoadFloat3(&m_vLocalTrailHigh);
XMStoreFloat3(&m_TrailVertices[m_iEndIndex].vPosition, XMVector3TransformCoord(vLocalTrailLow, vtxTrail_Low_Matrix));
XMStoreFloat3(&m_TrailVertices[m_iEndIndex + 1].vPosition, XMVector3TransformCoord(vLocalTrailHigh, vtxTrail_High_Matrix));
for (_uint i = 0; i < m_iCatmullRomCnt; ++i)
{
    _uint index = i * 2 + m_iVerticesCnt - 2;
    _float fWeight = (_float)(i + 1) / (m_iCatmullRomCnt + 1);
    _vector vPosHigh = XMVectorCatmullRom
    (
        XMVectorSetW(XMLoadFloat3(&m_TrailVertices[m_iCatmullRomIndex[0]].vPosition), 1.f),
        XMVectorSetW(XMLoadFloat3(&m_TrailVertices[m_iCatmullRomIndex[1]].vPosition), 1.f),
        XMVectorSetW(XMLoadFloat3(&m_TrailVertices[m_iCatmullRomIndex[2]].vPosition), 1.f),
        XMVectorSetW(XMLoadFloat3(&m_TrailVertices[m_iCatmullRomIndex[3]].vPosition), 1.f),
        fWeight
    );
    XMStoreFloat3(&m_TrailVertices[index].vPosition, vPosHigh);

    _vector vPosLow = XMVectorCatmullRom
    (
        XMVectorSetW(XMLoadFloat3(&m_TrailVertices[m_iCatmullRomIndex[0] + 1].vPosition), 1.f),
        XMVectorSetW(XMLoadFloat3(&m_TrailVertices[m_iCatmullRomIndex[1] + 1].vPosition), 1.f),
        XMVectorSetW(XMLoadFloat3(&m_TrailVertices[m_iCatmullRomIndex[2] + 1].vPosition), 1.f),
        XMVectorSetW(XMLoadFloat3(&m_TrailVertices[m_iCatmullRomIndex[3] + 1].vPosition), 1.f),
        fWeight
    );
    XMStoreFloat3(&m_TrailVertices[index + 1].vPosition, vPosLow);
}
m_iVerticesCnt = m_iEndIndex + 2;
```

Catmull-Rom 중간 정점 삽입

객체별 동적 버퍼와 Catmull-Rom 트레일 · 계속

```
m_iCatmullRomIndex[0] = m_iCatmullRomIndex[1];
m_iCatmullRomIndex[1] = m_iCatmullRomIndex[2];
for (_uint i = 0; i < m_iVerticesCnt; i += 2)
{
    m_TrailVertices[i].vTexcoord = { (_float)i / (_float)(m_iVerticesCnt - 2), 1.f }; //low
    m_TrailVertices[i+1].vTexcoord = { (_float)i / (_float)(m_iVerticesCnt - 2), 0.f }; //high
}
for (_uint i = m_iVerticesCnt; i < m_iNumMaxVertices-2; i += 2)
{
    m_TrailVertices[i].vTexcoord = { };
    m_TrailVertices[i+1].vTexcoord = { };
}
D3D11_MAPPED_SUBRESOURCE subResource;
m_pContext->Map(this->m_pVB, 0, D3D11_MAP_WRITE_DISCARD, 0, &subResource);
VTXPOSTEX* pVertices = (VTXPOSTEX*)subResource.pData;
for (_uint i = 0; i < m_iVerticesCnt; ++i)
{
    pVertices[i].vPosition = m_TrailVertices[i].vPosition;
    pVertices[i].vTexcoord = m_TrailVertices[i].vTexcoord;
}

m_pContext->Unmap(this->m_pVB, 0);

if(m_iVerticesCnt>=6)
    m_iNumIndices = 3 * 2 * (m_iVerticesCnt / 2 - 1);
return;
```

UV와 버텍스 버퍼 갱신

05 / 트레일과 화면 효과

파티클을 인스턴스 버퍼로 묶기

파티클별 제어값을 인스턴스 데이터로 전달하고 기하 셰이더에서 쿼드를 생성한다.

[블로그 원문 ↗](#)

구현 구조

TEXT

Engine/Private/VIBuffer_Particle_Instance.cpp
Client/Bin/ShaderFiles/Shader_VtxPointInstance.hlsl

개별 인스턴스 위치·수명 갱신 → 기본 정점과 인스턴스 버퍼 바인딩 → 인스턴스 드로우 → 셰이더에서 파티클 전개.

정점 버퍼와 인스턴스 버퍼 분리

기본 점을 담은 버퍼와 파티클마다 달라지는 행렬·수명 정보를 담은 버퍼를 입력 조립기에 함께 바인딩한다. 인스턴스 수를 드로우 호출에 전달해 같은 기본 형상을 반복해서 그린다.

Engine/Private/VIBuffer_Particle_Instance.cpp 발췌

CPP

```
HRESULT CVIBuffer_Particle_Instance::Render()
{
    m_pContext->DrawIndexedInstanced(m_iNumIndexPerInstance, m_iNumInstances, 0, 0, 0);
    return S_OK;
}
```

파티클 수명에 따라 위치 갱신

낙하형 파티클은 개별 속도로 Y 위치를 낮추고 경과 시간을 누적한다. 수명이 끝난 인스턴스는 반복 설정에 따라 초기 위치와 시간을 다시 사용한다. 갱신한 값은 인스턴스 버퍼에 기록한다.

파티클을 인스턴스 버퍼로 묶기 · 계속

Engine/Private/VIBuffer_Particle_Instance.cpp 발췌

```

CPP

void CVIBuffer_Particle_Instance::Drop(_float fTimeDelta)
{
    D3D11_MAPPED_SUBRESOURCE SubResource{};
    m_pContext->Map(m_pVBInstance, 0, D3D11_MAP_WRITE_NO_OVERWRITE, 0, &SubResource);
    VTXINSTANCE* pVertices = static_cast<VTXINSTANCE*>(SubResource.pData);
    for (size_t i = 0; i < m_iNumInstances; ++i)
    {
        pVertices[i].vTranslation.y -= m_pSpeeds[i] * fTimeDelta;
        pVertices[i].vLifeTime.y += fTimeDelta;
        if (pVertices[i].vLifeTime.y >= pVertices[i].vLifeTime.x)
        {
            if (true == m_isLoop) {
                pVertices[i].vTranslation = m_pInstanceVertices[i].vTranslation;
                pVertices[i].vLifeTime.y = 0.f;
            }
        }
    }
    m_pContext->Unmap(m_pVBInstance, 0);
}

```

셰이더에서 화면에 표시

포인트 인스턴스 셰이더는 입력된 위치와 축을 이용해 파티클 면을 만든다. 빌보드 경로와 비빌보드 경로를 구분하고, 픽셀 셰이더는 텍스처와 수명을 사용해 최종 표시를 결정한다.

파티클 실행과 버퍼 자료

```

struct VS_OUT_NON_BILLBOARD
{
    float4 vPosition : POSITION;
    row_major float4x4 TransformMatrix : WORLD;
    float2 vLifeTime : TEXCOORD0;
};

VS_OUT_NON_BILLBOARD VS_MAIN_NON_BILLBOARD(VS_IN In)
{
    VS_OUT_NON_BILLBOARD Out = (VS_OUT_NON_BILLBOARD) 0;
    matrix matWV, matWVP;

    matrix TransformMatrix = (matrix)0;
    TransformMatrix._11_22_33 = 1;
    TransformMatrix._41_42_43_44 = In.TransformMatrix._41_42_43_44;
    vector vPosition = mul(float4(In.vPosition, 1.f), TransformMatrix);

    Out.vPosition = mul(vPosition, g_WorldMatrix);

    Out.vLifeTime = In.vLifeTime;
    Out.TransformMatrix = In.TransformMatrix;
    return Out;
}

```

인스턴스 제어 정보를 월드 공간으로 옮기는 코드

```

struct GS_OUT
{
    float4 vPosition : SV_POSITION; //gs에서 빌보드하고 vp까지 곱해서 연산끝낼거
    float2 vTexcoord : TEXCOORD0;
    float2 vLifeTime : TEXCOORD1;
};

```

정점을 클립 공간으로 보내는 코드

파티클을 인스턴스 버퍼로 묶기 · 계속

```

[maxvertexcount(6)]
void GS_MAIN_NOM_BILLBOARD(point GS_IN_NOM_BILLBOARD In[1], inout TriangleStream<GS_OUT> OutStream)
{
    GS_OUT Out[4];

    float fRightSize = length(In[0].TransformMatrix._11,12,13);
    float fUpSize = length(In[0].TransformMatrix._21,22,23);
    vector vLook = In[0].TransformMatrix._31,32,33,34;
    float3 vRight = normalize(cross(float3(0.f, 1.f, 0.f), vLook.xyz)) * fRightSize;
    float3 vUp = normalize(cross(vLook.xyz, vRight)) * fUpSize;
    matrix matVP = mul(g_ViewMatrix, g_ProjMatrix);

    Out[0].vPosition = float4(In[0].vPosition.xyz + vRight + vUp, 1.f);
    Out[0].vPosition = mul(Out[0].vPosition, matVP);
    Out[0].vTexcoord = float2(0.f, 0.f);
    Out[0].vLifetime = In[0].vLifetime;

    Out[1].vPosition = float4(In[0].vPosition.xyz - vRight + vUp, 1.f);
    Out[1].vPosition = mul(Out[1].vPosition, matVP);
    Out[1].vTexcoord = float2(1.f, 0.f);
    Out[1].vLifetime = In[0].vLifetime;

    Out[2].vPosition = float4(In[0].vPosition.xyz - vRight - vUp, 1.f);
    Out[2].vPosition = mul(Out[2].vPosition, matVP);
    Out[2].vTexcoord = float2(1.f, 1.f);
    Out[2].vLifetime = In[0].vLifetime;

    Out[3].vPosition = float4(In[0].vPosition.xyz + vRight - vUp, 1.f);
    Out[3].vPosition = mul(Out[3].vPosition, matVP);
    Out[3].vTexcoord = float2(0.f, 1.f);
    Out[3].vLifetime = In[0].vLifetime;

    /* 카메라를 바라보는 포인트를 기준으로 4방향의 정점을 구한다-> 이 4개의 정점을 인덱스 매핑해서 6개의 정점으로 삼각형을 만든다.*/
    OutStream.Append(Out[0]);
    OutStream.Append(Out[1]);
    OutStream.Append(Out[2]);
    OutStream.RestartStrip(); //triangle stream 방식을 하지 않도록 삼각형 만들고 관음으로서 triangle_list 와같은 형태로 삼각형 정보를 채운다 */

    OutStream.Append(Out[0]);
    OutStream.Append(Out[2]);
    OutStream.Append(Out[3]);
    OutStream.RestartStrip();
}

```

기하 셰이더에서 파티클 쿼드 생성

```

void CVIBuffer_Particle_Instance::Spread_Gravity(_float fTimeDelta)
{
    D3D11_MAPPED_SUBRESOURCE SubResource{};
    m_pContext->Map(m_pVBInstance, 0, D3D11_MAP_WRITE_NO_OVERWRITE, 0, &SubResource);
    VTXINSTANCE* pVertices = static_cast<VTXINSTANCE*>(SubResource.pData);

    for (size_t i = 0; i < m_iNumInstances; ++i)
    {
        _vector vDir = XMLoadFloat4(&pVertices[i].vTranslation) - XMVectorSetW(XMLoadFloat3(&m_vPivot), 1.f);
        vDir = XMVector3Normalize(vDir);
        vDir = XMVectorSetY(vDir, m_pSpeeds_Gravity[i]);
        XMStoreFloat4(&pVertices[i].vTranslation, XMLoadFloat4(&pVertices[i].vTranslation) + vDir * fTimeDelta);
        m_pSpeeds_Gravity[i] -= m_fGravity * fTimeDelta*2.f;

        pVertices[i].vLifetime.y += fTimeDelta;

        if (pVertices[i].vLifetime.y >= pVertices[i].vLifetime.x)
        {
            if (true == m_isloop)
            {
                //다시유너래 상태로 되돌린다 (콜링저런)
                pVertices[i].vTranslation = m_pInstanceVertices[i].vTranslation;
                pVertices[i].vLifetime.y = 0.f;
                m_pSpeeds_Gravity[i] = m_pSpeeds[i];
            }
        }
    }

    m_pContext->Unmap(m_pVBInstance, 0);
}

```

파티클에 중력을 적용하는 코드



보스 공격 구체가 사라지며 파티클로 흩어지는 장면 · MP4 직접 열기

06 / 트레일과 화면 효과

노멀·깊이 렌더 타깃으로 카툰 외곽선 그리기

네 방향 이웃 픽셀의 노멀·깊이 차이를 계산하고 두 경계 조건을 함께 적용해 외곽선을 합성한다.

[블로그 원문 ↗](#)

구현 구조

TEXT

Engine/Bin/ShaderFiles/Shader_Deferred.hlsl

노멀·깊이 렌더 타깃 → 네 방향 이웃 샘플 → 차이 누적 → 두 임계값 비교 → 검은 외곽선 합성.

노멀 차이로 경계 찾기

노멀 텍스처의 값을 -1~1 범위로 복원한다. 네 방향의 이웃 노멀과 현재 노멀 사이의 차이를 누적하고 보정 계수를 곱해 경계 강도를 만든다.

Engine/Bin/ShaderFiles/Shader_Deferred.hlsl 발췌

HLSL

```
float DetectNormalEdges(float2 uv, float2 PixelSize)
{
    vector vCenterNormal = g_NormalTexture.Sample(LinearSampler, uv) * 2.f - 1.f;
    vector NormalOffsets[4];
    NormalOffsets[0] = g_NormalTexture.Sample(LinearSampler_Clamp, uv + float2(-PixelSize.x/5, 0.f)) * 2.f - 1.f;
    NormalOffsets[1] = g_NormalTexture.Sample(LinearSampler_Clamp, uv + float2(PixelSize.x/5, 0.f)) * 2.f - 1.f;
    NormalOffsets[2] = g_NormalTexture.Sample(LinearSampler_Clamp, uv + float2(0.f, -PixelSize.y/5)) * 2.f - 1.f;
    NormalOffsets[3] = g_NormalTexture.Sample(LinearSampler_Clamp, uv + float2(0.f, PixelSize.y/5)) * 2.f - 1.f;
    float fEdgeStrength = 0.0f;
    for (int i = 0; i < 4; ++i)
    {
        fEdgeStrength += length(vCenterNormal - NormalOffsets[i]);
    }
    return saturate(fEdgeStrength * 2.f);
}
```

깊이 경계와 함께 외곽선 결정

깊이 검출은 텍스처의 Y 성분에 1000을 곱해 비교한다. 최종 합성은 깊이 경계가 0.8 이상이고 노멀 경계가 0.2 이상인 픽셀에 외곽선을 적용한다. 이 패스의 샘플 간격은 1280×720을 기준으로 설정돼 있다.

노멀·깊이 렌더 타깃으로 카툰 외곽선 그리기 · 계속

Engine/Bin/ShaderFiles/Shader_Deferred.hlsl 발췌

HLSL

```
float DetectDepthEdges(float2 uv, float2 PixelSize)
{
    vector vCenterDepth = g_DepthTexture.Sample(LinearSampler, uv);
    float fViewZ = vCenterDepth.y * 1000.f;
    float DepthOffsets[4];
    DepthOffsets[0] = g_DepthTexture.Sample(LinearSampler_Clamp, uv + float2(-PixelSize.x, 0.f)).y*1000.f;
    DepthOffsets[1] = g_DepthTexture.Sample(LinearSampler_Clamp, uv + float2(PixelSize.x, 0.f)).y * 1000.f;
    DepthOffsets[2] = g_DepthTexture.Sample(LinearSampler_Clamp, uv + float2(0.f, -PixelSize.y)).y * 1000.f;
    DepthOffsets[3] = g_DepthTexture.Sample(LinearSampler_Clamp, uv + float2(0.f, PixelSize.y)).y * 1000.f;
    float fEdgeStrength = 0.0f;
    for (int i = 0; i < 4; ++i)
    {
        fEdgeStrength += length(fViewZ - DepthOffsets[i]);
    }
    return saturate(fEdgeStrength * 20.f);
}
```



지형과 캐릭터의 카툰 외곽선 · MP4 직접 열기

노멀·깊이 렌더 타깃으로 카툰 외곽선 그리기 · 계속



카툰 렌더링 결과 화면

```
//내부 외곽선 검출 ( 법선)
float DetectNormalEdges(float2 uv, float2 PixelSize)
{ //중심 픽셀의 깊이값
    vector vCenterNormal = g.NormalTexture.Sample(LinearSampler, uv) * 2.f - 1.f;

    vector NormalOffsets[4];
    NormalOffsets[0] = g.NormalTexture.Sample(LinearSampler Clamp, uv + float2(-PixelSize.x/5, 0.f)) * 2.f - 1.f;
    NormalOffsets[1] = g.NormalTexture.Sample(LinearSampler Clamp, uv + float2(PixelSize.x/5, 0.f)) * 2.f - 1.f;
    NormalOffsets[2] = g.NormalTexture.Sample(LinearSampler Clamp, uv + float2(0.f, -PixelSize.y/5)) * 2.f - 1.f;
    NormalOffsets[3] = g.NormalTexture.Sample(LinearSampler Clamp, uv + float2(0.f, PixelSize.y/5)) * 2.f - 1.f;

    //상하좌우 깊이 변화로 엣지 강도 계산
    float fEdgeStrength = 0.0f;
    for (int i = 0; i < 4; ++i)
    {
        fEdgeStrength += length(vCenterNormal - NormalOffsets[i]);
    }
    return saturate(fEdgeStrength * 2.f);
}
```

노멀 차이 계산

노멀·깊이 렌더 타깃으로 카툰 외곽선 그리기 · 계속

```
//내부 Depth 검출 깊이 ( )
float DetectDepthEdges(float2 uv, float2 PixelSize)
{ //중심 픽셀의 깊이값
    vector vCenterDepth = g_DepthTexture.Sample(LinearSampler, uv);

    float fViewZ = vCenterDepth.y * 1000.f;
    float DepthOffsets[4];
    DepthOffsets[0] = g_DepthTexture.Sample(LinearSampler_Clamp, uv + float2(-PixelSize.x, 0.f)).y * 1000.f;
    DepthOffsets[1] = g_DepthTexture.Sample(LinearSampler_Clamp, uv + float2(PixelSize.x, 0.f)).y * 1000.f;
    DepthOffsets[2] = g_DepthTexture.Sample(LinearSampler_Clamp, uv + float2(0.f, -PixelSize.y)).y * 1000.f;
    DepthOffsets[3] = g_DepthTexture.Sample(LinearSampler_Clamp, uv + float2(0.f, PixelSize.y)).y * 1000.f;
    //상하좌우 깊이 변화로 엣지 강도 계산
    float fEdgeStrength = 0.0f;
    for (int i = 0; i < 4; ++i)
    {
        fEdgeStrength += length(fViewZ - DepthOffsets[i]);
    }
    return saturate(fEdgeStrength * 20.f);
}
```

깊이 차이 계산

Engine/Bin/ShaderFiles/Shader_Deferred.hlsl · 합성 조건

```
if (0.8f <= DetectDepthEdges(In.vTexcoord, float2((1.f / 1280.f), (1.f / 720.f))) && 0.2f <= DetectNormalEdges(In.vTexcoord, float2((1.f / 1280.f), (1.f / 720.f))))
    Out.vFinal = float4(0.f, 0.f, 0.f, 1.f);
```

07 / 트레일과 화면 효과

다운샘플링과 분리 블러로 글로우 합성하기

발광 텍스처를 축소하고 가로·세로로 블러 처리한 뒤 원본 화면에 가산 합성한다.

[블로그 원문 ↗](#)

구현 구조

TEXT

Engine/Bin/ShaderFiles/Shader_Deferred.hlsl
Engine/Private/Renderer.cpp

발광 대상 렌더링 → 다운샘플링 → 가로·세로 블러 → 업샘플링 → 원본 화면에 더하기.

축을 나눠 블러 계산

축소한 발광 텍스처에 가로·세로 패스를 적용한다. 가로 패스는 X 좌표를 이동하며 13개 샘플에 가중치를 곱한다. 세로 패스는 같은 구성을 Y 방향으로 적용한다.

Engine/Bin/ShaderFiles/Shader_Deferred.hlsl 발췌

HLSL

```
PS_OUT PS_MAIN_DOWNSAMPLING_BLURX(PS_IN In)
{
    PS_OUT Out = (PS_OUT) 0;
    float2 vTexcoord = 0.f;
    for (int i = -6; i < 7; i++)
    {
        vTexcoord = float2(In.vTexcoord.x + (1.f / g_TexelSizeToSampling.x) * i, In.vTexcoord.y);
        vector vBlur = g_SamplingBlurXTexture.Sample(LinearSampler_Clamp, vTexcoord);
        vBlur.w = 0;
        Out.vColor += g_fWeights[i + 6] * vBlur;
    }
    Out.vColor /= 6.f;
    return Out;
}
```

확대 결과를 화면에 합성

업샘플링한 발광 색에 강도 텍스처의 값을 반영하고 원본 화면 색에 더한다. 이 합성 경로가 밝은 부분 주변으로 퍼지는 글로우를 만든다.

다운샘플링과 분리 블러로 글로우 합성하기 · 계속

Engine/Bin/ShaderFiles/Shader_Deferred.hlsl 발췌

```

HLSL
PS_OUT PS_MAIN_GLOW_DEFFERED_UPSAMPLING(PS_IN In)
{
    PS_OUT Out = (PS_OUT) 0;
    vector vGlowOriginColor = g_Glow_Original_Texture.Sample(LinearSampler, In.vTexcoord);
    vector vGlowColor = g_Glow_UpSampling.Sample(LinearSampler, In.vTexcoord);
    vector vGlowIntensity = g_Glow_Intensity_Texture.Sample(LinearSampler, In.vTexcoord);
    vGlowColor = Gaussian_Col_UpSampling(In.vTexcoord);
    vGlowIntensity.w = 0.f;
    if (0.f == vGlowIntensity.x &&
        dot(vGlowColor.xyz, vGlowColor.xyz) != 0
    )
        vGlowIntensity.x = 3.f;
    else
        vGlowIntensity.x *= 6.f;
    Out.vColor = vGlowOriginColor + vGlowColor * vGlowIntensity.x;
    return Out;
}

```



글로우 결과 화면

```

PS_OUT PS_MAIN_DOWNSAMPLING (PS_IN In)//9
{
    PS_OUT Out = (PS_OUT) 0;
    float4 vColor = g_SamplingTexture.Sample(LinearSampler, In.vTexcoord);
    Out.vColor = vColor;
    return Out;
}

```

다운샘플링

다운샘플링과 분리 블러로 글로우 합성하기 · 계속

```

PS_OUT PS_MAIN_DOWNSAMPLING_BLURX(PS_IN In)//10
{
    PS_OUT Out = (PS_OUT) 0;
    float2 vTexCoord = 0.f;
    for (int i = -6; i < 7; i++)
    {
        vTexCoord = float2(In.vTexCoord.x + (1.f / g_TexelSizeToSampling.x) * i, In.vTexCoord.y);
        vector vBlur = g_SamplingBlurXTexture.Sample(LinearSampler_Clamp, vTexCoord);
        vBlur.w = 0;
        Out.vColor += g_fWeights[i + 6] * vBlur;
    }
    Out.vColor /= 6.f;
    return Out;
}

PS_OUT PS_MAIN_DOWNSAMPLING_BLURY(PS_IN In)//11
{
    PS_OUT Out = (PS_OUT) 0;
    float2 vTexCoord = 0.f;
    vector vBlur = 0.f;
    for (int i = -6; i < 7; i++)
    {
        vTexCoord = float2(In.vTexCoord.x, In.vTexCoord.y + (1.f / g_TexelSizeToSampling.y) * i);
        vector vBlurY = g_SamplingBlurYTexture.Sample(LinearSampler_Clamp, vTexCoord);
        vBlurY.w = 0;
        vBlur += g_fWeights[i + 6] * vBlurY;
    }
    vBlur /= 6.f;
    /*x 축 블러 적용한 파이널 텍스처에 적용*/
    Out.vColor = vBlur;
    return Out;
}

```

가로·세로 블러

```

PS_OUT PS_MAIN_UPSAMPLING(PS_IN In)//12
{
    PS_OUT Out = (PS_OUT) 0;
    vector vColor = g_Glow_UpSampling.Sample(LinearSampler, In.vTexCoord);
    Out.vColor = vColor;
    return Out;
}

```

업샘플링

08 / 트레일과 화면 효과

노이즈 임계값과 경계 색으로 디졸브 구현하기

시간에 따른 노이즈 UV와 임계값을 사용해 픽셀을 제거하고 사라지는 경계에 색을 적용한다.

[블로그 원문 ↗](#)

구현 구조

TEXT

Client/Bin/ShaderFiles/Shader_VtxAnimMesh.hlsl

노이즈 UV 계산 → 노이즈와 임계값 비교 → 픽셀 제거 → 경계 색 출력.

시간 비율로 노이즈 UV 이동

`PS_MAIN_DESOLVE()`는 현재 노이즈 시간과 최대 시간의 비율을 구한다. 노이즈의 RG 값으로 UV 오프셋을 만들고 강도와 시간 비율을 곱한다. 변형한 UV에서 읽은 노이즈 값을 디졸브 기준으로 사용한다.

임계값 아래의 픽셀 제거

노이즈 값이 임계값보다 작으면 `discard`로 픽셀을 제거한다. 임계값 주변에서는 `smoothstep`으로 경계 색을 계산해 사라지는 면의 가장자리를 표시한다.

Client/Bin/ShaderFiles/Shader_VtxAnimMesh.hlsl 발췌

```
HLSL

PS_OUT PS_MAIN_DESOLVE(PS_IN In)
{
    PS_OUT Out = (PS_OUT) 0;
    vector vMtrlDiffuse = g_DiffuseTexture.Sample(LinearSampler, In.vTexcoord);
    if(vMtrlDiffuse.a<0.1f)
        discard;
    Out.vDiffuse = vMtrlDiffuse;
    float fRatio = (g_NoiseCurTime / g_NoiseMaxTime);
    vector vNoise = g_NoiseTexture.Sample(LinearSampler, In.vTexcoord);
    float2 UVDistortion = (vNoise.rg - 0.5f) * g_NoiseStrength * fRatio;
    float2 DistortedUV = clamp(In.vTexcoord + UVDistortion, 0.f, 1.f);
    vector vDistortedNoise = g_NoiseTexture.Sample(LinearSampler, DistortedUV);
    float Alpha_ToDiscard = vDistortedNoise.r;
    if(Alpha_ToDiscard <fRatio)
    {
        discard;
    }
    else if( Alpha_ToDiscard>= fRatio -0.2f && Alpha_ToDiscard<= fRatio+0.2f)
    {
        float GlowFactor = smoothstep(fRatio - 0.02f, fRatio + 0.02f, Alpha_ToDiscard);
        Out.vDiffuse.r = 1.0f + GlowFactor * 2.0f;
        Out.vDiffuse.g = Out.vDiffuse.r * 0.2f;
        Out.vDiffuse.b = Out.vDiffuse.r * 0.1f;
        Out.vDiffuse.rgb *= GlowFactor * g_GlowIntensity;
    }
}
```

노이즈 임계값과 경계 색으로 디졸브 구현하기 · 계속

HLSL · 계속

```

    }
    else
    {
        float GlowFactor = smoothstep(fRatio - 0.05f, fRatio + 0.05f, Alpha_ToDiscard);
        Out.vDiffuse.rgb += GlowFactor * g_GlowIntensity;
        Out.vDiffuse.rgb *= lerp(1.f, 0.f, fRatio);
    }
    Out.vNormal = float4(In.vNormal.xyz * 0.5f + 0.5f, 0.f);
    Out.vDepth = float4(In.vProjPos.z / In.vProjPos.w, In.vProjPos.w / 1000.f, 0.f, 0.f);
    Out.vWorld = In.vWorldPos;
    return Out;
}

```



몬스터 사망 시 적용한 디졸브 효과

```

if (Alpha_ToDiscard < fRatio)
{
    discard;
}

else if (Alpha_ToDiscard >= fRatio - 0.2f && Alpha_ToDiscard <= fRatio + 0.2f)
{
    float GlowFactor = smoothstep(fRatio - 0.02f, fRatio + 0.02f, Alpha_ToDiscard);
    Out.vColor.r = 1.0f + GlowFactor * 2.0f;
    Out.vColor.g = Out.vColor.r * 0.2f;
    Out.vColor.rgb *= GlowFactor * g_GlowIntensity;
}
else
{
    float GlowFactor = smoothstep(fRatio - 0.05f, fRatio + 0.05f, Alpha_ToDiscard);
    Out.vColor.rgb += GlowFactor * g_GlowIntensity;
    Out.vColor.rgb *= lerp(1.f, 0.f, fRatio);
}

```

노이즈 임계값과 경계 색 계산

09 / 트레일과 화면 효과

화면 UV와 노이즈로 물 디스토션 구현하기

투영 위치에서 화면 UV를 구하고 굴절 방향·시간 기반 노이즈를 더해 물 뒤의 화면을 다시 샘플링한다.

[블로그 원문 ↗](#)

구현 구조

TEXT

Client/Bin/ShaderFiles/Shader_VtxWater.hlsl

투영 위치를 화면 UV로 변환 → 굴절 방향과 노이즈 오프셋 계산 → 화면 텍스처 재샘플링 → 물 색과 혼합.

물 표면 뒤의 화면 좌표 구하기

`PS_MAIN_WAVE()`는 투영 위치를 `W`로 나눈 뒤 `X·Y`를 0~1 범위의 화면 UV로 바꾼다. 시선 방향과 노멀에서 굴절 방향을 구하고, 노이즈에 시간 기반 `sin·cos`를 곱해 샘플 위치를 움직인다.

화면 색과 물 텍스처 혼합

화면 텍스처는 굴절 방향과 노이즈 오프셋을 더한 UV로 읽는다. 물의 디퓨즈 색과 화면 색을 0.5 비율로 섞고 디스토션 출력에 기록한다.

Client/Bin/ShaderFiles/Shader_VtxWater.hlsl 발췌

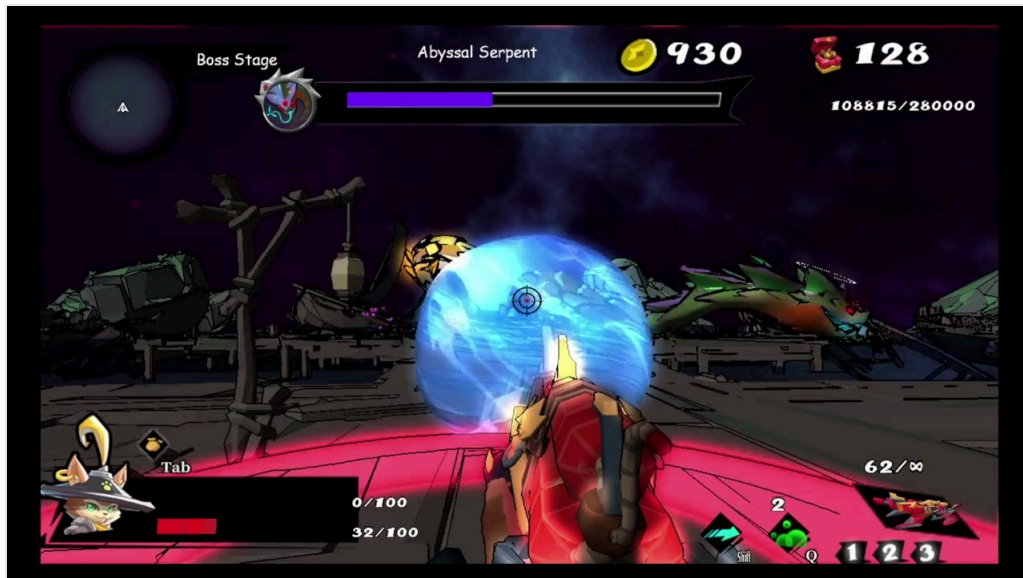
HLSL

```
PS_WAVE_OUT PS_MAIN_WAVE(PS_WAVE_IN In)
{
    PS_WAVE_OUT Out = (PS_WAVE_OUT) 0;
    float3 vViewDir = normalize(In.vWorldPos - g_vCamPosition);
    float3 RefractionDir = refract(vViewDir, In.vNormal.xyz, g_Refractive_index);
    float2 vTexcoordScreen;
    vTexcoordScreen.x = In.vProjPos.x / In.vProjPos.w;
    vTexcoordScreen.y = In.vProjPos.y / In.vProjPos.w;
    vTexcoordScreen.x = (vTexcoordScreen.x) * 0.5f + 0.5f;
    vTexcoordScreen.y = (vTexcoordScreen.y) * -0.5f + 0.5f;
    float2 vTexcoordNoise = In.vTexcoord;
    float3 vNoise = g_NoiseTexture.Sample(LinearSampler, vTexcoordNoise);
    float2 noiseOffset = (vNoise.gb - 0.5f) * g_NoiseStrength;
    noiseOffset.x = vNoise.x * g_NoiseStrength * cos(g_Time*0.8f );
    noiseOffset.y = vNoise.y * g_NoiseStrength * sin(g_Time *0.8f);
    float2 Refraction_Texcoord = RefractionDir.xy*0.1f + noiseOffset + vTexcoordScreen;
    float2 Diffuse_ReflectedTexcoord = RefractionDir.xy * 0.5f + noiseOffset + In.vTexcoord;
    vector vDiffuseColor = g_DiffuseTexture.Sample(LinearSampler, Diffuse_ReflectedTexcoord);
    vDiffuseColor.a = 0.5;
    vector RefracedColor = g_ScreenTexture.Sample(LinearSampler, Refraction_Texcoord);
    vector vFinalColor = lerp(vDiffuseColor, RefracedColor, vDiffuseColor.a);
    vFinalColor.a = 0.5f;
    Out.vDistortion = vFinalColor;
    return Out;
}
```

화면 UV와 노이즈로 물 디스토션 구현하기 · 계속

HLSL · 계속

}



보스 전투의 물 구체 디스토션 · MP4 직접 열기

```
//노이즈 샘플링
float2 vTexCoordNoise = In.vTexCoord;
float3 vNoise = g_NoiseTexture.Sample(LinearSampler, vTexCoordNoise);
//노이즈 move -
float2 noiseOffset = (vNoise.gb*0.1f - 0.1f) * g_NoiseStrength;
noiseOffset.x = vNoise.x * g_NoiseStrength * sin(g_Time* 3.f);
noiseOffset.y = vNoise.y * g_NoiseStrength * sin(g_Time * 3.f);
//스크린 겹치는 스크린 샘플링
float2 Refraction_Texcoord = vRefractDir.x*0.1f + noiseOffset + vTexCoordScreen;
//스크린 겹치는 스크린 샘플링
vector RefractedColor = g_ScreenTexture.Sample(LinearSampler, Refraction_Texcoord);
```

노이즈 오프셋으로 화면 텍스처 샘플링

10 / 애니메이션과 리소스

현재 자세에서 다음 애니메이션으로 전환하기

진행 중인 동작을 끝 프레임으로 되돌리지 않고 현재 뼈 자세를 새 보간의 시작점으로 사용한다.

[블로그 원문 ↗](#)

구현 구조

TEXT

```
Engine/Private/Animation.cpp
Engine/Private/Channel.cpp
Engine/Private/Model.cpp
```

전환 시점 자세 저장 → 목표 키프레임 준비 → 뼈별 SRT 보관 → 로컬 행렬 갱신.

전환 시작 자세 보관

`Store_CurrentAnim_KeyFrame_State()`는 전환을 시작하는 시점의 채널 값을 뼈 인덱스로 보관한다. 이미 전환 중이면 보관 중인 자세에서 다시 시작 키를 구성한다. 목표 애니메이션의 키와 같은 뼈 인덱스로 연결해 자세를 전환한다.

이동·크기는 Lerp, 회전은 Slerp

트랙 위치와 전환 길이로 보관 비율을 구한다. 크기와 이동은 `XMVectorLerp()`, 쿼터니언 회전은 `XMQuaternionSlerp()`로 계산한다. 결과를 하나의 아핀 행렬로 조립해 뼈에 전달한다.

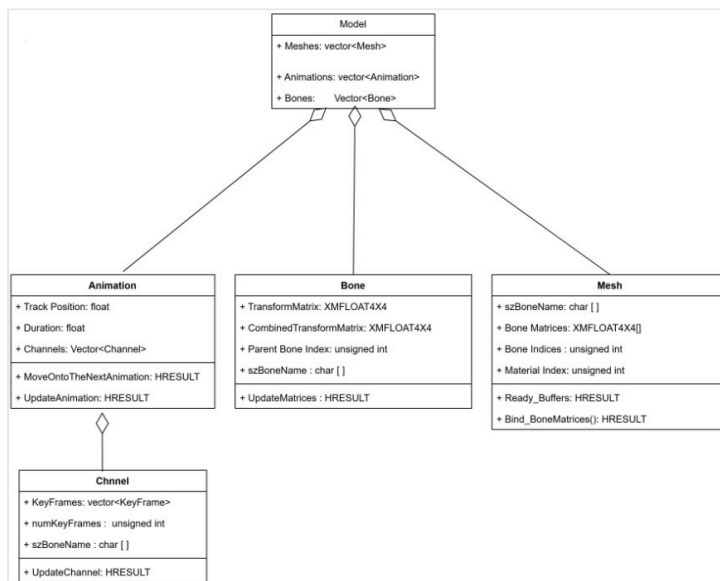
현재 자세에서 다음 애니메이션으로 전환하기 · 계속

Engine/Private/Animation.cpp 발췌

CPP

```
_bool CAnimation::Update_TransformMatrix_ForMotionChange(const vector<class CBone*>& Bones, _uint iBoneIndex, KEYFRAME Start, KEYFRAME Last)
{
    if ((m_fCurrentTrackPosition - m_KeyFramePosOfStartAnim) > m_fDistance)
        return true;
    _float fRatio = (m_fCurrentTrackPosition - m_KeyFramePosOfStartAnim) / m_fDistance;
    _float3 vScale;
    _float3 vPosition;
    _float4 vRotation;
    _float3 vLeftScale, vRightScale;
    _float4 vLeftRotation, vRightRotation;
    _float3 vLeftPosition, vRightPosition;
    vLeftScale = Start.vScale;
    vLeftRotation = Start.vRotation;
    vLeftPosition = Start.vPosition;
    vRightScale = Last.vScale;
    vRightRotation = Last.vRotation;
    vRightPosition = Last.vPosition;
    XMStoreFloat3(&vScale, XMVectorLerp(XMLoadFloat3(&vLeftScale), XMFLOAT3(&vRightScale), fRatio));
    XMStoreFloat4(&vRotation, XMQuaternionSlerp(XMFLOAT4(&vLeftRotation), XMFLOAT4(&vRightRotation), fRatio));
    XMStoreFloat3(&vPosition, XMVectorLerp(XMFLOAT3(&vLeftPosition), XMFLOAT3(&vRightPosition), fRatio));
    Bones[iBoneIndex]->Set_TransformationMatrix(XMMatrixAffineTransformation(XMFLOAT3(&vScale), XMVECTORSET(0.f, 0.f, 0.f, 1.f), XMFLOAT4(&vRotation), XMVECTORSET(XMFLOAT3(&vPosition), 1.f)));
    return false;
}
```

전환 장면과 애니메이션 자료



모델·애니메이션·채널·키 프레임 구조

현재 자세에서 다음 애니메이션으로 전환하기 · 계속

```

/*For Next Anim*/
void CModel::Move_NextAnim(_uint NextAnimIndex,_float fDistanceAnimToAnim ,_bool isNextAnimationLoop, _bool IsOnTrackPosition )//애니메이션이 진행중인가? //트랙포지션에 있는가?
{
    m_Animations[m_iCurrentAnimIndex]->Store_CurrentAnim_KeyFrame_State(m_Bones, IsOnTrackPosition);//(뼈 객체 들을 저장한 컨테이너, 현재 애니메이션이 진행중인지 여부)
    m_Animations[m_iCurrentAnimIndex]->Store_AnimToChange_KeyFrame_State(m_Bones, m_Animations[NextAnimIndex]);//(뼈 객체 들을 저장한 컨테이너, 다음 애니메이션 객체)
    m_Animations[m_iCurrentAnimIndex]->SetUp_DistanceAnimToAnim(fDistanceAnimToAnim); //현재 애니메이션 상태에서 다음 애니메이션으로 갈때 보간 Duration 간격
    m_iNextAnimIndex = NextAnimIndex;
    m_isNextAnimationLoop = isNextAnimationLoop;
    m_isChangingMotion = true;
}

```

목표 애니메이션 설정 코드

```

void CAnimation::Store_AnimToChange_KeyFrame_State(const vector<class CBone*>& Bones, CAnimation* NextAnim)
{
    if (m_mapStartKeyFrames.empty())
    {
        MSG_BOX("Store CurrentAnimState Before Changing Motion");
        return;
    }

    m_mapLastKeyFrames.clear();
    m_KeyFramePosOfNextAnim = m_fDistance+m_KeyFramePosOfStartAnim;

    for (auto &CurKeyFramePair: m_mapStartKeyFrames)
    {
        KEYFRAME KeyFrame = {};
        _uint BoneIndex = { 0 };
        if(NextAnim->Get_FrameInfo(KeyFrame,CurKeyFramePair.first)) // 다음 애니메이션과 겹치는 부위일때만 추가하자
            m_mapLastKeyFrames.emplace(CurKeyFramePair.first,KeyFrame);
    }
}

```

공통 뼈 채널의 시작 키 프레임 보관

```

void CAnimation::Store_CurrentAnim_KeyFrame_State(const vector<class CBone*>& Bones ,_bool IsOnTrackPosition)
{
    if (IsOnTrackPosition)
    {
        if (m_mapStartKeyFrames.empty())
        {
            MSG_BOX("Store CurrentAnimState Before Changing Motion");
            return;
        }
        for (auto& FramePair : m_mapStartKeyFrames)
        {
            KEYFRAME keyFrame = {};
            if (m_mapLastKeyFrames.end() != m_mapLastKeyFrames.find(FramePair.first)) {
                _uint iBoneIndex = FramePair.first;
                Set_KeyFrames_OnTrackPos(Bones, m_mapStartKeyFrames[iBoneIndex], m_mapLastKeyFrames[iBoneIndex], keyFrame);
                m_mapStartKeyFrames[FramePair.first] = keyFrame;
            }
        }
    }
}

```

진행 중인 애니메이션의 현재 키 프레임 취득

현재 자세에서 다음 애니메이션으로 전환하기 · 계속

```
bool CAnimation::Update_TransformationMatrices_ForMotionChange(const vector<class CBone*>& Bones, _bool isLoop, _float fTimeDelta)
{
    m_szName;

    m_fCurrentTrackPosition += m_fTickPerSecond * fTimeDelta * m_fTickRatio;

    _bool IsFinished = true;
    _bool bReturn = true;
    for (auto&LastKeyFramePair : m_mapLastKeyFrames)
    {
        _uint iBoneKey = LastKeyFramePair.first;
        IsFinished = Update_TransformMatrix_ForMotionChange(Bones, iBoneKey, m_mapStartKeyFrames[iBoneKey], LastKeyFramePair.second);
        if (false == IsFinished)
            bReturn = false;
    }
}
```

전환 중 트랙 위치 갱신과 뼈별 변환 함수 호출



무기 동작이 이어지는 애니메이션 전환 시연 · [MP4 직접 열기](#)

11 / 애니메이션과 리소스

UI 태그 등록과 팝업 Z 순서·이벤트 관리

UI 매니저가 활성 상태와 팝업 깊이를 관리하고, UI 객체가 입력 상태에 연결된 콜백을 실행한다.

[블로그 원문 ↗](#)

구현 구조

TEXT

Engine/Private/UI_Manager.cpp
Engine/Private/UIObject.cpp

태그로 UI 등록 → 활성화와 팝업 깊이 갱신 → 깊이 순서 정리 → UI 상태별 이벤트 실행.

팝업 활성화와 Z 순서

UI 매니저는 태그로 객체를 찾고 활성화한다. 팝업은 현재 최대 깊이에 1을 더한 값을 받아 다른 팝업보다 앞쪽 순서로 이동한다. 고정 UI는 등록 시 깊이 10을 사용한다.

Engine/Private/UI_Manager.cpp 발췌

CPP

```
HRESULT CUI_Manager::Set_Active_A_UI(const _wstring UI_Tag)
{
    CUIObject* pInstance = { nullptr };
    if (nullptr == (pInstance = Find_UIObject(UI_Tag)) )
    {
        MSG_BOX("There's no UI you named like that");
    }
    pInstance->Set_Active();
    if (UI_TYPE::UI_POP == pInstance->Get_Type())
    {
        pInstance->Set_SortDepth(++m_iMaxPopUpUIDepth);
    }
    return S_OK;
}
```

팝업 깊이를 연속된 값으로 정리

Update_UIs()는 깊이가 작은 UI부터 꺼내는 우선순위 큐를 만든다. 팝업의 상대 순서를 유지하면서 최소 팝업 깊이부터 번호를 다시 부여하고 최대 깊이를 갱신한다.

UI 태그 등록과 팝업 Z 순서·이벤트 관리 · 계속

Engine/Private/UI_Manager.cpp 발췌

```

CPP
void CUI_Manager::Update_UIs()
{
    _uint m_iCurIndex = m_iMinPopUpUIDepth;
    priority_queue< pair< _uint, CUIObject*> , vector<pair<_uint,CUIObject*>> , greater<pair<_uint, CUIObject*>>> p
q;
    for (auto iter = m_mapUIs.begin(); iter != m_mapUIs.end(); ++iter)
    {
        pq.push({ iter->second->Get_SortDepth(), iter->second });
    }
    while (!pq.empty())
    {
        pair<_uint, CUIObject*> pObjectInfo =pq.top();
        if(pObjectInfo.second->Get_Type() == UI_TYPE::UI_POP)
        {
            if (pObjectInfo.first != m_iCurIndex)
            {
                pObjectInfo.second->Set_SortDepth(m_iCurIndex);
            }
            ++m_iCurIndex;
        }
        pq.pop();
    }
    if(m_iMinPopUpUIDepth != m_iCurIndex)
        m_iMaxPopUpUIDepth = m_iCurIndex - 1;
}

```

입력 상태와 콜백 연결

CUIObject는 마우스 좌표와 UI 사각형의 포함 여부를 검사한다. Update_Event()는 UI 상태에 해당하는 함수가 등록돼 있으면 호출한다. 버튼의 이벤트 동작을 상태별 함수 배열에 연결하는 구조다.

Engine/Private/UIObject.cpp 발췌

```

CPP
void CUIObject::Update_Event()
{
    for (_uint i = 0; i < (_uint)UI_STATE::STATE_END; ++i)
    {
        if (i == (_uint)m_eUIState)
        {
            if (nullptr!=m_EventFunction[i])
            {
                m_EventFunction[(uint)m_eUIState]();
            }
        }
    }
}

```

UI 태그 등록과 팝업 Z 순서·이벤트 관리 · 계속



NPC 상호작용과 무기 UI 열기·닫기 · MP4 직접 열기

12 / 애니메이션과 리소스

FBX 모델과 애니메이션을 실행용 바이너리로 변환하기

Assimp가 읽은 정적·애니메이션 모델의 메시, 재질, 뼈와 키 데이터를 정해진 순서로 쓰고 런타임 로더가 다시 조립한다.

[블로그 원문 ↗](#)

구현 구조

TEXT

```
Engine/Private/Model.cpp
Engine/Private/AI_Info.cpp
```

Assimp 모델 읽기 → 메시·재질·뼈·키프레임 기록 → 같은 순서로 바이너리 쓰기 → 런타임 모델 조립.

모델에서 실행에 필요한 데이터를 꺼내기

맵과 몬스터의 모델에는 메시·재질·텍스처 경로가 들어 있고, 움직이는 모델에는 뼈와 애니메이션 정보도 들어 있다. 실행할 때마다 원본 FBX의 구조를 해석하는 과정을 줄이기 위해, 필요한 데이터를 바이너리 파일로 기록하는 코드를 작성했다.

변환은 `CModel::Make_Binary_NonAnim()`과 `CModel::Make_Binary_Anim()`에서 수행한다. 두 함수는 입력 모델 경로와 출력 파일 경로를 받아 Assimp가 읽은 데이터를 `WriteFile()`로 기록한다. 런타임의 `AI_Info`는 `ReadFile()`로 같은 순서의 데이터를 읽어 메시·재질·애니메이션 구조를 구성한다.

Assimp로 읽고 출력 파일 열기

정적 모델 변환 함수는 좌수 좌표계 변환과 실시간 처리용 플래그를 설정한다. `TYPE_NONANIM`에는

`aiProcess_PreTransformVertices`를 추가한다. 원본 모델 읽기가 실패하면 파일 기록으로 넘어가지 않는다.

CPP

```
_uint iFlag = {
    aiProcess_ConvertToLeftHanded | aiProcessPreset_TargetRealtime_Fast
};

if (eModelType == TYPE_NONANIM)
    iFlag |= aiProcess_PreTransformVertices;

pAIScene = Importer.ReadFile(pModelFilePath, iFlag);
if (0 == pAIScene)
    return E_FAIL;

HANDLE hFile = CreateFile(OutputFilePath, GENERIC_WRITE, 0, 0,
    CREATE_ALWAYS, FILE_ATTRIBUTE_NORMAL, 0);
```

출력 파일은 `CREATE_ALWAYS`로 연다. 같은 출력 경로로 다시 변환하면 파일 내용도 새로 기록된다. 변환 함수는 출력 핸들을 확인한 뒤 메시 개수부터 순서대로 쓴다.

FBX 모델과 애니메이션을 실행용 바이너리로 변환하기 · 계속

정적 메시: 개수와 데이터 순서를 맞추기

파일 앞에 메시 개수를 쓰고, 각 메시마다 재질 인덱스·정점 수·면 수를 기록한다. 읽는 코드는 이 개수로 배열 크기와 반복 횟수를 정한다.

파일 영역	기록하는 데이터	읽는 쪽의 역할
메시 개수	<code>mNumMeshes</code>	메시 배열 크기 설정
메시별 정보	재질 인덱스, 정점 수, 면 수, 이름 길이와 이름 영역	메시 구성과 반복 횟수 설정
정점	위치, 노멀, UV, 탄젠트	각 정점 속성 배열 구성
면	삼각형의 정점 인덱스 3개	삼각형 연결 정보 구성
재질	재질 수, 텍스처 타입별 개수, 경로 길이와 경로	재질별 텍스처 경로 구성

정적 모델의 위치에는 `XMVector3TransformCoord()`로 `PreTransformMatrix`를 적용한다. 노멀과 탄젠트에는 `XMVector3TransformNormal()`을 사용한다. 위치와 방향을 구분해 변환한 뒤 파일에 쓴다.

정점 하나를 쓰는 순서는 다음과 같다.

CPP

```
WriteFile(hFile, &vPosition, sizeof(_float3), &dwByte, nullptr);
WriteFile(hFile, &vNormal, sizeof(_float3), &dwByte, nullptr);
WriteFile(hFile, &vTexcoord, sizeof(_float2), &dwByte, nullptr);
WriteFile(hFile, &vTangent, sizeof(_float3), &dwByte, nullptr);
```

`AI_Info::Initialize_NonAnim()`도 같은 순서와 크기로 읽는다.

CPP

```
ReadFile(hFile, &vPosition, sizeof(_float3), &dwByte, nullptr);
ReadFile(hFile, &vNormal, sizeof(_float3), &dwByte, nullptr);
ReadFile(hFile, &vTexcoord, sizeof(_float2), &dwByte, nullptr);
ReadFile(hFile, &vTangent, sizeof(_float3), &dwByte, nullptr);

MeshInfo.m_vecPositions[j] = vPosition;
MeshInfo.m_vecNormals[j] = vNormal;
MeshInfo.m_vecTexcoords[j] = vTexcoord;
MeshInfo.m_vecTangents[j] = vTangent;
```

파일 안에는 각 정점 필드의 이름이 따로 들어 있지 않다. 따라서 쓰기 함수와 읽기 함수가 순서와 자료형 크기를 함께 지켜야 한다. 한쪽에서 필드를 추가하면 뒤에 있는 데이터의 읽기 위치도 달라진다.

FBX 모델과 애니메이션을 실행용 바이너리로 변환하기 · 계속

애니메이션 메시: 뼈와 정점의 관계 보존

움직이는 모델에는 정점 배열만으로 부족하다. 어떤 뼈가 어떤 정점에 얼마만큼 영향을 주는지도 남겨야 한다.

`Make_Binary_Anim()`은 메시의 기본 정점 데이터 뒤에 뼈 개수와 뼈별 정보를 추가하고, 그다음 면 인덱스를 쓴다.

뼈마다 오프셋 행렬과 이름, 가중치 개수를 기록한다. 각 가중치는 영향을 받는 정점 ID와 가중치 값으로 구성된다.

CPP

```
WriteFile(hFile, &ID, sizeof(_uint), &dwByte, nullptr);
WriteFile(hFile, &Weight, sizeof(_float), &dwByte, nullptr);
```

정점 ID는 메시 안에서 영향을 받는 정점을 지정하고, 가중치는 해당 뼈의 변환이 정점에 반영되는 비율을 지정한다.

`AI_Info::Initialize_Anim()`은 이 데이터를 읽어 메시의 뼈 정보에 보관한다.

노드 계층: 자식 수와 재귀 순회로 구조 복원

메시가 참조하는 뼈 정보와 노드의 부모-자식 구조는 함께 필요하다. `Write_Bones_Anim()`은 루트 노드부터 이름, 변환 행렬, 자식 수를 기록하고 각 자식을 재귀적으로 방문한다. 노드의 변환 행렬은 전치한 뒤 저장한다.

CPP

```
_int iNumChildren = pNode->mNumChildren;
WriteFile(hFile, &iNumChildren, sizeof(_int), &dwByte, nullptr);

for (int i = 0; i < iNumChildren; ++i)
{
    Write_Bones_Anim(hFile, pNode->mChildren[i]);
}
```

읽는 쪽의 `AI_Info::Read_Bones()`는 노드 하나를 만들고 이름과 행렬, 자식 수를 읽는다. 자식 배열을 그 수만큼 확보한 뒤 같은 순서로 재귀 호출한다. 노드마다 자식 수를 기록했기 때문에 다음 바이트가 자식 노드인지, 해당 하위 계층을 마친 다음 데이터인지 읽는 코드가 구분할 수 있다.

애니메이션 채널: 값과 시간을 함께 저장

노드 계층 다음에는 애니메이션 개수를 기록한다. 각 애니메이션은 이름, `Duration`, `TickPerSecond`, 채널 수를 가진다. 채널에는 대상 노드 이름과 위치·크기·회전 키 배열이 들어간다.

위치와 크기 키의 값은 `_float3`, 회전 키의 값은 `_float4`로 기록한다. 각 값 뒤에는 `_double` 시간값을 붙인다. 위치 키 기록 부분은 다음과 같다.

FBX 모델과 애니메이션을 실행용 바이너리로 변환하기 · 계속

CPP

```
_uint iNumPositionsKeys = pChannel->mNumPositionKeys;
WriteFile(hFile, &iNumPositionsKeys, sizeof(_uint), &dwSize, nullptr);
for (int k = 0; k < iNumPositionsKeys; ++k)
{
    _float3 vPos = {};
    _double mTime = { 0 };
    memcpy(&vPos, &(pChannel->mPositionKeys[k].mValue), sizeof(_float3));
    memcpy(&mTime, &(pChannel->mPositionKeys[k].mTime), sizeof(_double));
    WriteFile(hFile, &vPos, sizeof(_float3), &dwByte, nullptr);
    WriteFile(hFile, &mTime, sizeof(_double), &dwByte, nullptr);
}
```

값만 저장하면 어느 시점의 자세인지 알 수 없다. 시간값과 `TickPerSecond`를 함께 보관해야 런타임에서 재생 시점에 해당하는 키를 선택하고 보간할 수 있다. 읽어 들인 채널과 키는 [애니메이션 전환](#)에도 사용한다.

13 / 애니메이션과 리소스

작업 함수와 스레드로 리소스 로딩 나누기

리소스 등록 함수를 여러 로더에 전달하고 작업별 스레드와 완료 플래그로 로딩을 구성한다.

[블로그 원문 ↗](#)

구현 구조

TEXT

```
Client/Private/Loader.cpp
Client/Private/AsyncLoader.cpp
Client/Public/AsyncLoader.h
```

리소스 등록 작업을 함수로 전달 → 작업별 스레드 시작 → 함수 실행 → 완료 플래그 검사 → 전체 로딩 완료.

등록 작업을 함수 객체로 전달

CLoader는 모델·텍스처·게임 오브젝트 등록을 람다로 묶어 여러 로더에 전달한다. `Begin_Thread()`는 전달받은 `std::function<HRESULT()>`를 저장하고 `_beginthreadex()`로 실행 진입점을 시작한다.

Client/Private/AsyncLoader.cpp 발췌

CPP

```
HRESULT CAsyncLoader::Begin_Thread(std::function<HRESULT()> pEntryPoint)
{
    m_pFunction = std::bind(pEntryPoint);
    InitializeCriticalSection(&m_CriticalSection);
    m_hThread = (HANDLE)_beginthreadex(nullptr, 0, ALoadingMain, this, 0, nullptr);
    if (0 == m_hThread)
        return E_FAIL;
    return S_OK;
}
```

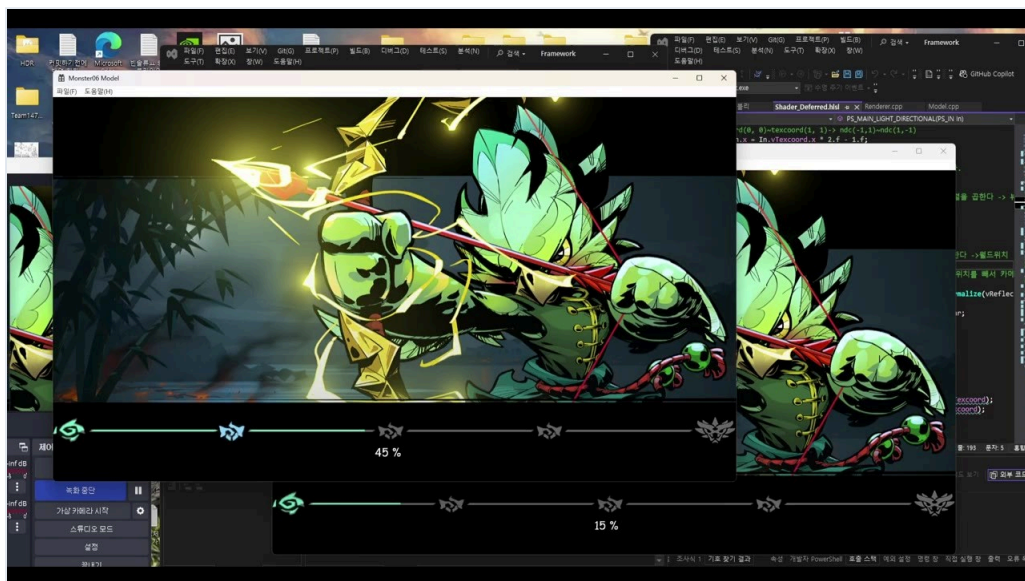
스레드 진입점에서 로딩 실행

진입점은 전달받은 로더의 `Loading()`을 호출한다. 로딩 함수가 리소스 등록 작업을 수행하고, 상위 로더는 각 작업의 `isFinished()`를 검사해 전체 완료 상태를 설정한다. 작업 함수의 반환값과 작업 종료 표시는 각각 `HRESULT`와 완료 플래그로 표현한다.

작업 함수와 스레드로 리소스 로딩 나누기 · 계속

Client/Private/AsyncLoader.cpp 발췌

```
CPP
_uint APIENTRY ALoadingMain(void* pArg)
{
    CAsyncLoader* pLoader = static_cast<CAsyncLoader*>(pArg);
    if (FAILED(pLoader->Loading())) {
        return 1;
    }
    return 0;
}
```



작업 단위로 나눈 리소스 로딩 시연 · MP4 직접 열기

모델 파일의 기록·읽기 순서는 [모델 바이너리](#)에서 다룬다.